

5. 検索・いいね編

実装する機能

機能	実装内容
公開動画検索	条件未指定時は公開動画一覧、条件指定時は通常検索を行い、通常検索が0件の場合は定義済み条件で類似検索へ切り替える
タイトル検索	大文字・小文字を区別しない部分一致検索を行う
カテゴリー検索	定義済みカテゴリーとの完全一致で検索する
条件未指定検索	titleとcategoryが未指定の場合は、公開条件を満たす動画をすべて新しい順で取得する
通常検索	titleは大文字・小文字を区別しない部分一致、categoryは完全一致で検索する
類似検索Fallback	titleを含む通常検索が0件の場合に限り、titleに近い公開動画を類似度順で取得する
複合検索	タイトルとカテゴリーの両方を指定した場合はAND条件で絞り込む
Cursorページング	検索条件を維持したまま次ページを取得する
いいね状態表示	ログイン中のUser本人がいいね済みかを表示する
いいね件数表示	公開動画に紐づく現在のいいね件数を表示する
いいね	activeなログインUserが公開動画へいいねする
いいね解除	ログインUserが自分のいいね関係を解除する
Frontend画面	検索画面といいね操作をブラウザから利用できるようにする
OpenAPI	Query、Response、Status、認証、エラー契約を固定する
テスト	Entity、Migration、Repository、Usecase、Validator、Controller、Frontend、結合、ブラウザ確認を行う

本編で確定する仕様

検索一致方式と、いいね再送時の扱いを本編で確定する。

未確定だった項目	本編での確定内容	理由
タイトル一致方式	大文字・小文字を区別しない部分一致	タイトルの先頭以外に含まれる「焙煎」「ラテアート」などの語でも探せるようにし、検索要件を実用的に満たすため
タイトル検索索引	<code>pg_trgm</code> と公開条件付きGIN索引を使用する	<code>%検索語%</code> の全件走査を避けるため
条件未指定時	公開条件を満たす動画をすべて取得	検索画面を開いた直後から動画を表示するため
通常一致	titleは大文字・小文字を区別しない部分一致、categoryは完全一致とする	既存の検索仕様を維持する。
類似検索の発動条件	titleを含む通常結果が0件の場合だけ実行する	一致する動画がある場合に無関係な候補を混在させない。
類似度関数	<code>word_similarity(lower(title_query), lower(videos.title))</code> を使用する	検索語と動画タイトル全体を単純比較するのではなく、検索語に最も近い連続部分をタイトル内から判定できるため
類似検索Operator	<code>lower(title_query) <% lower(videos.title)</code> を使用する	<code>pg_trgm</code> のGIN Indexを利用して類似候補を絞り込めるため

未確定だった項目	本編での確定内容	理由
類似度の基準	<code>pg_trgm.word_similarity_threshold</code> はPostgreSQLの既定値 <code>0.6</code> を使用し、Coffee Reelから別の値へ変更しない	PostgreSQLが <code>word_similarity</code> 用に定めている既定値を使用し、根拠のない独自値を追加しないため
category指定時の類似検索	指定categoryを維持した範囲でtitleの類似検索を行う	利用者が指定したカテゴリ条件を無視しないため
類似結果の並び順	<code>word_similarity(lower(title_query), lower(videos.title)) DESC</code> 、 <code>videos.created_at DESC</code> 、 <code>videos.id DESC</code>	最も近い動画を先にし、類似度が同じ場合も順序を固定するため
類似候補も0件	<code>result_type = similar</code> と空配列を返す	類似検索を実行した事実をFrontendへ伝えながら、無関係な動画へ差し替えないため
検索結果の種別	Responseへ <code>result_type</code> を all(検索条件で未指定の公開動画一覧)と matched(通常検索に一致した結果), similar(通常検索0件後の類似した結果)に固定して返す	Frontが一致した結果と類似した結果を推測せずに表示できるようにするため。
検索Route	既存の <code>GET /videos</code> を使用する	条件なしは公開ルール、条件ありは検索として扱い、同じ公開動画取得処理を重複させないため
いいね操作Response	<code>PUT /videos/:video_id/like</code> と <code>DELETE /videos/:video_id/like</code> は、 <code>200 OK</code> で更新後の <code>video_id</code> 、 <code>like_count</code> 、 <code>is_liked</code> を返す。 <code>GET /videos/:video_id/like</code> は作成しない	いいね操作後の追加GETを不要にし、1回のAPI通信でDB確定後の件数と本人状態へ同期するため。重複いいねと存在しないいいね解除も、現在状態を返して成功扱いにできる
いいね解除Response	<code>DELETE /videos/:video_id/like</code> は <code>200 OK</code> で更新後の <code>video_id</code> 、 <code>like_count</code> 、 <code>is_liked = false</code> を返す	追加通信を行わず、削除後の正しい件数へ同期するため
幕等な再送	重複いいね、存在しないいいね解除も <code>200 OK</code> で現在状態を返す	通信再送や画面状態のずれを安全に処理するため
いいね状態取得Route	独立した <code>GET /videos/:video_id/like</code> は作成しない	一覧・詳細の初期状態は既存Response、操作後の状態はPUT・DELETE Responseで取得できるため

前提条件

本編の実装前に、次が完成していること。

前提	使用目的
User Entity	Role、Status、IDを確認する
Video Entity	処理状態、公開状態、削除状態を確認する
CategoryCode	検索カテゴリーを定義済み値へ限定する
usersテーブル	いいね操作Userの現在Statusを確認する
videosテーブル	タイトル、カテゴリー、公開状態、削除状態を検索する
video_output metasテーブル	公開動画の再生URL・サムネイルURL発行に必要な内部Keyを取得する
saved_videosテーブル	既存の <code>is_saved</code> を検索結果へ含める
ObjectStorageRepository	期限付き再生URL・サムネイルURLを発行する
公開動画一覧API	<code>GET /videos</code> で公開動画を取得できる
公開動画詳細API	<code>GET /videos/:video_id</code> で公開動画詳細を取得できる
Optional Auth Middleware	ゲストとログインUserの公開取得を分ける
Auth Middleware	activeなログインUserだけをいいねAPIへ通す

前提	使用目的
AuthContext	Frontendでログイン状態を共有する
共通API Client	Access Token、共通エラー、Token再発行を扱う
共通Cursor処理	Backend生成の不透明Cursorを検証する
管理者・投稿管理編	<code>hidden</code> 動画を検索結果・新規いいね対象から除外する
管理者・ユーザー管理編	<code>suspended</code> Userの認証必須操作を拒否する

公開動画の共通条件は、既存の動画編で定義した `ready / published / 未削除` を使用する。検索用に別の公開条件を作らない。

要件上の機能順

要件定義に記載された順番を維持する。

1. 動画検索
2. いいね
3. いいね解除

いいねは要件上の優先順位が検索より低いため、検索を完成させてから実装する。

実際のコード実装順

Backend

1. 既存の公開動画取得条件、Optional Auth、Cursor仕様を確認する。
2. 部分一致検索、類似Fallback条件、類似度基準、並び順、Cursor、いいね冪等性、API Statusを確認する。
3. `VideoLike` Entityを実装する。
4. 検索・いいね用Domain Errorを確認・追加する。
5. `video_likes` Go Migrationを実装する。
6. `pg_trgm`、公開タイトル検索索引、類似検索で使用する索引をGo Migrationで実装する。
7. 既存 `VideoRepository` Interfaceへ公開検索条件を追加する。
8. `VideoLikeRepository` Interfaceを定義する。
9. 条件未指定一覧と通常検索QueryをGORMで実装する。
10. 通常検索0件時に使用する類似検索QueryをGORMで実装する。
11. `VideoLikeRepository` をGORMで実装する。
12. いいね作成と更新後Like状態取得を同じRepositoryメソッド内で実装する。
13. いいね解除と更新後Like状態取得を同じRepositoryメソッド内で実装する。
14. 既存 `VideoUsecase` へ検索条件を接続する。
15. 既存 `VideoUsecase` へ通常検索と類似Fallback判定を接続する。
16. 既存 `VideoValidator` へ検索Query検証を追加する。
17. いいね用Video ID検証を既存Validatorへ接続する。
18. 既存 `VideoController.ListReels` へ検索Queryを接続する。
19. `VideoLikeController` を実装する。

20. RouterへいいねAPIを接続する。
21. `main.go` でRepository、Usecase、Controllerを接続する。
22. OpenAPIへ検索Query、いいねAPI、Response項目を追加する。
23. Backendテストを実行する。
24. API単体で検索といいねを確認する。
25. 管理者非公開・User停止との結合確認を行う。

Frontend

1. 既存 `types/video.ts` へ `like_count` と `is_liked` を追加する。
2. 既存 `api/video.ts` へ検索条件を追加する
3. `api/video_like.ts` を作成する
4. `SearchPage` を作成する
5. 検索フォームとURL Queryを接続する
6. 検索結果一覧とCursorページングを接続する
7. `result_type = similar` の場合の案内を表示する
8. 類似候補も0件の場合は空結果を表示する
9. `LikeButton` を作成する
10. リールへいいね表示・操作を接続する
11. 動画詳細へいいね表示・操作を接続する
12. 検索結果へいいね表示・操作を接続する
13. React Routerへ検索画面を接続する
14. ナビゲーションへ検索導線を追加する
15. Frontendテストを実行する
16. BackendとFrontendを起動する
17. ゲスト、User、Admin、suspended Userでブラウザ確認する

実装停止条件

次のいずれかが未解決の場合は、Frontend実装へ進まない。

- 公開条件がRepositoryごとに別々の意味で実装されている
- タイトル検索をSQL文字列連結で実装している
- 部分一致検索に必要な索引がない
- Cursorが検索条件変更後もそのまま使用される
- 検索結果取得時に動画ごとのLike COUNTでN+1 Queryが発生する
- 重複いいねを事前存在確認だけで防いでいる
- 管理者非公開といいね作成の競合を防げない
- suspended Userがいいねを作成できる
- private、hidden、削除済みVideoへ新規いいねを作成できる

- Like件数をVideoの可変カウンターだけで管理している
- SQL、制約名、Object Key、署名付きURLをエラーへ含めている
- 「近い動画」と判定する類似度基準が確定していない
- title未指定時にも類似検索を実行する実装になっている
- category指定時の類似検索でcategory条件が外れている
- matched結果へsimilar結果を混在させている
- similar結果をCreatedAtだけでページングしている
- Cursorへresult_typeと類似度が含まれていない
- Frontendがresult_typeを推測している
- 類似候補0件時に無条件の全動画へ差し替えている

完成状態

本編完成後、次を確認できること。

1. ゲストがタイトルから公開動画を検索できる。
2. ゲストがカテゴリーから公開動画を検索できる。
3. タイトルとカテゴリーを同時に指定して検索できる。
4. タイトルは大文字・小文字を区別しない部分一致になる。
5. 条件未指定のGET /videosは、公開条件を満たす動画を新しい順で返す。
6. 通常検索に一致する動画がある場合は、一致した動画だけを返す。
7. titleを含む通常検索が0件の場合は、定義済み基準を満たす類似動画を返す。
8. category指定時の類似検索でも、指定category条件を維持する。
9. 類似候補も0件の場合は空配列を返し、無条件の全動画へ差し替えない。
10. Responseの結果result_typeはall、matched、similarのいずれかになる。
11. matched結果はCreatedAt降順、同日時はID降順になる。
12. similar結果は類似度降順、同一類似度はCreatedAt降順、ID降順になる。
13. private動画が検索結果へ表示されない。
14. hidden動画が検索結果へ表示されない。
15. 削除済み動画が検索結果へ表示されない。
16. processing、failed等の未公開動画が検索結果へ表示されない。
17. 検索結果の動画へ期限付き再生URLとサムネイルURLだけを返す。
18. ゲストにも `like_count` を表示できる。
19. ゲストの `is_liked` は常にfalseになる。
20. ログインUserには本人の `is_liked` が返る。
21. activeなUserだけが公開動画へいいねできる。
22. 同じUserとVideoのいいね関係が2件作成されない。
23. 同じいいねRequestの再送は200 OKとなり、現在のlike_countとis_liked = trueが返る。

24. いいね解除で関係行が物理削除される。
25. 存在しないいいね解除の再送も200 OKとなり、現在のlike_countとis_liked = falseが返る。
26. いいね操作後に別の状態取得APIを呼ばず、PUT・DELETEのResponseだけで画面を同期できる。
27. hidden、private、削除済み動画へ新規いいねできない。
28. 管理者が動画をhiddenへ変更しても既存Like行を削除しない。
29. hidden動画をpublishedへ戻すと、既存Like件数を再利用できる。
30. suspended UserはいいねAPIを利用できない。
31. 同時いいねでもUNIQUE制約違反を外部へ露出しない。
32. SQL Injection用文字列を検索しても意図しないSQLが実行されない。
33. `%`、`_`、`\` は検索ワイルドカードではなく入力文字として扱える。
34. Backend、Frontend、結合テストが成功する。
35. ChromeとSafariで検索・いいねを確認できる。

全体フロー

公開動画検索

1. 利用者が検索画面を開く。
2. FrontendがURL Queryから現在の検索条件を復元する。
3. 利用者がタイトル、カテゴリーを入力する。
4. Frontendが前後空白を除去し、検索Requestを作成する。
5. Frontendが `GET /videos` へ `title`、`category`、`limit`、`cursor` を送る。
6. Optional Auth MiddlewareがAuthorization Headerを確認する。
7. Headerがなければゲストとして継続する。
8. Headerがある場合はAccess Tokenを通常どおり検証する。
9. 不正Tokenはゲスト扱いにせず401を返す。
10. ControllerがQueryを固定構造体へBindする。
11. Validatorがtitle、category、limit、cursorを検証・正規化する。
12. UsecaseがCursorと現在の検索条件が一致することを確認する。
13. Repositoryが公開条件を必ず適用する。
14. titleとcategoryが未指定の場合は、公開動画をCreatedAt降順、ID降順で取得する。
15. titleまたはcategoryが指定されている場合は、通常検索を実行する。
16. title指定時はEscape済み部分一致条件を追加する。
17. category指定時は完全一致条件を追加する。
18. 両方指定時はAND条件で取得する。
19. 通常検索結果が1件以上の場合は、result_type = matchedとして返す。
20. 通常検索結果が0件でtitleが指定されている場合は、類似検索を実行する。
21. categoryも指定されている場合は、同じcategory内だけを類似検索する。

22. `lower(title_query) <= lower(videos.title)` を条件として、`word_similarity` が0.6以上の動画を取得する。
23. 類似結果を `word_similarity DESC`、`CreatedAt DESC`、`ID DESC` で並べる。
24. 類似検索を実行した場合は、取得件数に関係なく `result_type = similar` として返す。
25. 類似候補が0件の場合は、`result_type = similar` と空配列を返す。
26. RepositoryがAuthor、OutputMeta、Like件数、本人Like状態、本人保存状態を一括取得する。
27. UsecaseがHasMoreと次Cursorを生成する。
28. Usecaseが公開対象Object Keyだけから期限付きURLを発行する。
29. Controllerが公開動画一覧Responseを返す。
30. Frontendがresult_typeに応じて一覧表示を切り替える。
31. result_type = similarの場合は「一致する動画が見つからなかったため、近い動画を表示しています」と表示する。
32. itemsが0件の場合は「該当する動画はありません」と表示する。

いいね

1. activeなログインUserが公開動画のいいねボタンを押す。
2. Frontendが `PUT /videos/:video_id/like` を呼ぶ。
3. Auth MiddlewareがAccess Token、User Status、TokenVersionを確認する。
4. Controllerが認証User IDとVideo IDを取得する。
5. ValidatorがVideo IDを検証する。
6. Usecaseがいいね処理をRepositoryへ依頼する。
7. RepositoryがDB Transactionを開始する。
8. RepositoryがUser行を共有ロックし、現在もactiveであることを確認する。
9. RepositoryがVideo行を共有ロックし、現在も公開条件を満たすことを確認する。
10. Repositoryが `video_likes` へ関係行を作成する。
11. 同じUserとVideoが既に存在する場合は、重複作成せず成功扱いにする。
12. Repositoryが現在のLike件数を取得する。
13. Repositoryがvideo_id、like_count、is_liked = trueをUsecaseへ返す。
14. TransactionをCommitする。
15. Controllerが更新後状態を200 OKで返す。
16. FrontendがResponseのlike_countとis_likedをそのまま画面へ反映する。
17. Frontendは追加の状態取得APIを呼ばない。

いいね解除

1. ログインUserがいいね済みボタンを押す。
2. Frontendが `DELETE /videos/:video_id/like` を呼ぶ。
3. Auth MiddlewareがUserを確認する。
4. ControllerがUser IDとVideo IDを取得する。
5. ValidatorがVideo IDを検証する。

6. RepositoryがDB Transactionを開始する。
7. RepositoryがVideoを共有ロックし、現在も公開条件を満たすことを確認する。
8. Repositoryがuser_idとvideo_idを条件にLike関係を物理削除する。
9. 削除件数が0件でも成功扱いにする。
10. Repositoryが削除後のLike件数を取得する。
11. Repositoryがvideo_id、like_count、is_liked = falseを返す。
12. TransactionをCommitする。
13. Controllerが更新後状態を200 OKで返す。
14. FrontendがResponseをそのまま画面へ反映する。
15. Frontendは追加の状態取得APIを呼ばない。

Clean Architectureの責務

層	責務
Entity	VideoLikeの不変条件と、公開Video判定に使用する既存Videoルールを保持する
DB	UserとVideoのLike関係、複合一意、外部キー、検索索引を保証する
Repository	GORM Query、部分一致検索、集計、存在状態、行ロック、用途別Transactionを扱う
Usecase	検索条件、公開取得、Cursor、URL発行、いいね業務フローを組み立てる
Controller	Echo Context、Path、Query、認証User、HTTP Status、Responseを扱う
Middleware	Optional AuthとAuthを適用し、suspended Userを拒否する
Validator	title、category、limit、cursor、Video IDの形式を検証・正規化する
Router	URL、HTTP Method、Middleware、Controllerを接続する
Frontend API	検索Queryといいね通信を扱う
React Page	検索フォーム、検索結果、ページングを表示する
React Component	Like件数、Like状態、Like操作を表示する

禁止事項

- 検索専用のSearch Entityを作らない。
- 検索専用テーブルを作らない。
- 既存の公開動画取得と別の公開条件を作らない。
- User入力をSQL文字列、列名、ORDER BYへ直接連結しない。
- User入力の `%` と `_` を意図せずワイルドカードとして扱わない。
- `any` や `map[string]interface{}` を使用しない。
- ControllerでDB検索、COUNT、Like作成を行わない。
- Usecaseへ `gorm.DB` を渡さない。
- UsecaseでBegin、Commit、Rollbackを扱わない。
- 汎用的な `TxManager` を作らない。
- Videoへ `LikeCount` を永続化して関係行と二重管理しない。
- いいね作成を事前存在確認だけで重複防止しない。

- hidden変更時にLike行を削除しない。
- published復帰時にLike行を再作成しない。
- Object Key、Storage Credential、署名付きURLをDBのLike行へ保存しない。
- いいねボタンへ `dangerouslySetInnerHTML` を使用しない。
- ゲストのLike操作でいいねAPIを呼ばない。

ファイル構成

Backend

```

backend/
├── entity/
│   ├── user.go
│   ├── video.go
│   ├── video_like.go
│   └── errors.go
├── db/
│   └── db.go
├── migrate/
│   ├── videos.go
│   ├── video_output metas.go
│   ├── saved_videos.go
│   ├── video_likes.go
│   └── video_search.go
├── repository/
│   ├── object_storage_repository.go
│   ├── video_repository.go
│   └── video_like_repository.go
├── usecase/
│   ├── video_usecase.go
│   └── video_like_usecase.go
├── validator/
│   └── video_validator.go
├── controller/
│   ├── video_controller.go
│   └── video_like_controller.go
├── middleware/
│   ├── auth_middleware.go
│   └── optional_auth_middleware.go
├── router/
│   └── router.go
└── main.go

```

検索は既存Videoの公開取得責務であるため、`search_repository.go`、`search_usecase.go`、`search_controller.go` は作らない。

既に適用済みの `videos.go` Migrationを書き換えず、タイトル検索拡張と索引は追加Migrationである `video_search.go` へ分ける。

Frontend

```

frontend/src/
├── api/
│   ├── client.ts
│   ├── video.ts
│   └── video_like.ts
├── components/
│   ├── LikeButton.tsx
│   ├── ReelVideo.tsx
│   └── VideoCard.tsx
├── pages/
│   ├── ReelPage.tsx
│   ├── VideoDetailPage.tsx
│   └── SearchPage.tsx
├── router/
│   └── AppRouter.tsx
├── types/
│   └── video.ts
└── App.tsx

```

同じ画面でしか使用しない検索フォーム処理を不要に別Hookへ分割しない。

LikeButtonはリール、詳細、検索結果で共通利用するためComponentへ分ける。

Entity仕様

VideoLike

役割

どのUserがどのVideoへいいねしたかという関係を表す。

Like件数はVideoLike行の件数から算出し、Entityへ可変カウンターとして保持しない。

フィールド

フィールド	型	仕様
ID	uint64	Like関係を識別するID。作成時にDBが発行するため、永続化前の新規Entityでは0、取得済みEntityでは正数として扱う
UserID	uint64	LikeしたUser ID。0を許可しない
VideoID	uint64	Like対象Video ID。0を許可しない
CreatedAt	time.Time	Like関係作成日時。UTCで扱い、未設定を許可しない

不変条件

- UserIDを0にしない。
- VideoIDを0にしない。
- 新規生成時はUserID、VideoID、CreatedAtを設定する。
- 作成済みのUserIDとVideoIDを変更しない。
- 更新操作を持たない。
- 解除時はEntityを更新せず、関係行を物理削除する。

- Like対象が公開Videoかどうかは、Video EntityとRepository Transactionで確認する。
- Userがactiveかどうかは、User EntityとRepository Transactionで確認する。

生成方法

既存Entity仕様の `NewVideoLike(userID, videoID, now)` を使用する。

Usecaseが認証Userと対象Videoを指定し、Repository Transactionが現在のUser StatusとVideo公開状態を最終確認して保存する。

Video

既存Video Entityを使用する。

検索専用EntityやLikeCountフィールドは追加しない。

公開検索と新規いいね対象は、既存の `CanBeViewedPublicly` と同じ条件だけを使用する。

条件	必須値
ProcessingStatus	<code>ready</code>
PublishStatus	<code>published</code>
DeletedAt	<code>nil</code>

DB仕様

使用するテーブル

テーブル	用途
users	Author情報とLike操作UserのStatusを確認する
videos	Title、Category、公開状態、削除状態を検索する
video_output metas	再生・サムネイル用の内部Object Keyを取得する
video_likes	UserとVideoのLike関係、Like件数を取得する
saved_videos	既存の <code>is_saved</code> を検索結果へ付与する

video_likes

役割

ログインUserとVideoのいいね関係を保存する。

video_likesの物理定義は、VideoLike EntityのGORMタグと本編のDB仕様を正本とする。

場所	責務
VideoLike Entity	主キー、NOT NULL、UserIDとVideoIDの複合UNIQUE、JSON出力制御
Migration	EntityをGORM Migratorへ渡して <code>video_likes</code> を作成する
<code>video_search.go</code>	<code>pg_trgm</code> の有効化と公開条件付きGIN Indexを作成する
Repository	行ロック、Transaction、幂等作成、削除後件数取得を行う
Usecase	検索条件、Cursor、いいね業務フローを組み立てる

保存する内容

項目	内容
ID	いいね関係を識別するID
UserID	いいねしたUser ID
VideoID	いいね対象のVideo ID
CreatedAt	いいねを作成した日時

保存ルール

- UserIDとVideoIDの組み合わせを重複させない。
- activeな認証Userだけが新規いいねできる。
- `ready / published / 未削除` のVideoだけを新規いいね対象とする。
- 同じUserとVideoへの再送では、新しい関係行を作らず成功扱いにする。
- 重複防止は、VideoLike Entityに定義したUserIDとVideoIDの複合UNIQUE制約を最終防衛線とする。
- 事前存在確認だけで重複防止しない。
- いいね解除時は関係行を物理削除する。
- Videoの論理削除、`private` 化、`hidden` 化では既存のLike行を削除しない。
- Videoが再公開された場合は、保持していたLike行を件数へ再利用する。
- VideoへLikeCountを保存して二重管理しない。
- Like件数は `video_likes` の関係行から算出する。

Entity・Migration・Repositoryの分担

本編では、いつLikeを作成・削除できるか、重複Requestをどう扱うか、非公開時に既存Likeをどう扱うかを定義する。

次の物理DB設計はDB仕様書へ記載する。

- PostgreSQL型
- NULL制約
- 外部キー
- ON DELETE
- UNIQUE制約
- Index名
- Index対象カラム

検索用Migration

検索・いいね編で確定した次の内容を、本編のMigration仕様として実装する。

- `pg_trgm` を `IF NOT EXISTS` で有効化する。
- `lower(title)` を対象とした公開条件付きGIN Indexを作成する。
- `idx_videos_public_title_trgm` を部分一致検索と類似検索の両方で使用する。
- 類似検索専用の重複Indexは作成しない。
- `pg_trgm.word_similarity_threshold` はPostgreSQLの既定値である `0.6` を使用する。

- 追加Migrationの `video_search.go` では、`pg_trgm` の有効化と `idx_videos_public_title_trgm` の作成だけを行う。
- 既存の公開Feed用IndexとCategory用Indexは再作成しない。

タイトル検索

タイトルは大文字・小文字を区別しない部分一致とする。

概念上の検索条件は次のとおりとする。

```
lower(title) LIKE '%' + lower(escaped_title) + '%' ESCAPE '\'
```

実装では検索文字列をSQLへ直接連結せず、GORMのプレースホルダーへ渡す。

類似検索

類似検索は、titleを含む通常の部分一致検索が0件の場合だけ実行する。

検索語を第1引数、動画タイトルを第2引数として `word_similarity` を使用する。

概念上の検索条件は次のとおりとする。

```
lower(title_query) <% lower(videos.title)
```

類似度の取得は、

```
word_similarity(lower(title_query), lower(videos.title))
```

エスケープ規則

入力文字	扱い
<code>%</code>	部分一致の任意文字列ではなく、文字 <code>%</code> として検索する
<code>_</code>	任意1文字ではなく、文字 <code>_</code> として検索する
<code>\</code>	Escape文字として二重化し、文字 <code>\</code> として検索する

pg_trgm

`video_search.go` Migrationで次を実行する。

1. `pg_trgm` を `IF NOT EXISTS` で有効化する
2. `idx_videos_public_title_trgm` を作成する

検索索引

名前	種別	対象・条件	用途
<code>idx_videos_public_title_trgm</code>	GIN	<code>lower(title) gin_trgm_ops</code> WHERE公開条件	Title部分一致検索と <code>word_similarity</code> による類似検索

公開条件は次の3条件へ統一する。

```
processing_status = ready
publish_status = published
deleted_at IS NULL
```

検索条件と並び順

`all` と `matched` には検索語との類似度順を使用しない。

`similar` の場合だけ、`word_similarity` による類似度順を使用する。

Like件数順とランダム順は追加しない。

result_type	並び順
all	<code>videos.created_at DESC</code> 、 <code>videos.id DESC</code>
matched	<code>videos.created_at DESC</code> 、 <code>videos.id DESC</code>
similar	<code>word_similarity(lower(title_query), lower(videos.title)) DESC</code> 、 <code>videos.created_at DESC</code> 、 <code>videos.id DESC</code>

title	category	結果
未指定	未指定	既存の公開リール一覧
指定	未指定	Title部分一致の公開Video
未指定	指定	Category完全一致の公開Video
指定	指定	Title部分一致かつCategory一致の公開Video

検索結果の並び順は結果種別ごとに次へ固定する。

通常一致がある場合は、従来どおり新着順を維持する。類似検索へ切り替わった場合だけ類似度順にする。

Like件数順とランダム順は追加しない。

Like件数

- VideoテーブルへLike件数カラムを追加しない。
- `video_likes` の関係行を正本とする。
- 公開一覧・検索・詳細では集約SubqueryまたはGROUP BYを使用し、VideoごとのN+1 COUNTを行わない。
- Likeが0件の場合は0を返す。
- private、hidden、削除済みVideo自体を一般Responseへ返さない。
- 再公開後は保持していたLike行を再び件数へ含める。

本人Like状態

利用者	is_liked
ゲスト	false
activeなログインUser	<code>video_likes.user_id = 認証User ID</code> が存在する場合true
不正Token	401。ゲストへ降格しない
suspended User	401。ゲストへ降格しない

ログインUserの状態は、公開一覧QueryでUser IDを固定値として渡し、1件ごとの存在確認を行わない。

Transaction・排他制御

共通方針

汎用的な `TxManager` は作成しない。

LikeとUnlikeは、それぞれ用途別のRepositoryメソッドをTransaction境界とする。

Likeでは、User状態確認、Video公開状態確認、Likeの冪等作成、作成後Like件数取得を同じTransaction内で行う。

Unlikeでは、Video公開状態確認、本人Likeの冪等削除、削除後Like件数取得を同じTransaction内で行う。
独立したLike状態取得処理は作成しない。

Like作成Transaction

同じTransaction内で次を行う。

1. Userを `FOR SHARE` で取得する。
2. Userが存在し、現在もactiveであることを確認する。
3. Videoを `FOR SHARE` で取得する。
4. Videoが `ready / published / 未削除` であることを確認する。
5. VideoLikeを作成する。
6. 複合UNIQUE競合は重複Likeとして成功扱いにする。
7. 全処理成功時だけCommitする。
8. DB失敗時はRollbackする。

ロック順

Like作成は必ず次の順番で取得する。

1. users
2. videos

User利用停止処理が `users → refresh_tokens → videos` の順でロックするため、Like側が `videos → users` で取得するとデッドロックの原因になる。

Videoの管理者非公開処理はVideoを更新ロックする。

Like側のVideo共有ロックと競合するため、次のどちらか一方が先に確定する。

先に確定した処理	結果
Like作成	公開中にLikeが作成され、その後hiddenになる。Like行は保持する
管理者非公開	Videoがhiddenになり、後続Like作成は公開条件違反で拒否される

これにより、hidden確定後に新しいLike行を作成しない。

いいね解除

Unlikeは次の条件で物理削除する。

```
user_id = 認証User ID
video_id = Path Video ID
```

- 他UserのLike行は削除できない。
- 0件削除は成功扱いにする。

いいね解除は、現在もready / published / 未削除のVideoだけを対象とする。

private、hidden、削除済み、処理未完了、存在しないVideoは、一般公開対象外として404 Not Foundへ統一する。

非公開中の既存Like行は削除せず保持する。Videoが再公開された後に、通常のいいね解除APIから解除できる。

いいね解除と削除後Like件数取得は、同じ用途別Repositoryメソッド内で処理する。

Transactionへ含めない処理

次をLike作成Transaction中に実行しない。

- Object Storage通信
- 期限付きURL発行
- Frontend通知
- 外部HTTP通信
- 構造化アクセスログ出力待ち

Repository仕様

VideoRepository Interface

既存Interfaceへ公開一覧・検索用の条件構造体を接続する。

公開検索条件

項目	型	内容
Title	string	Validatorによるtrimと文字数検証が完了した検索語。SQL LIKE用Escapeはまだ行わない。未指定時は空文字
Category	CategoryCode または未指定を表す明示型	定義済みCategory。未指定時はカテゴリーで絞り込まない
Limit	int	1~100
Cursor	固定Cursor構造体	CreatedAt、ID、FilterHash
ViewerUserID	*uint64	ゲストはnil、ログインUserは認証User ID
SearchMode	固定型	all,matched,similarのいずれか

Frontendから SearchMode を指定させない。通常検索が0件だったかどうかをUsecaseが判断し、Repositoryへ類似検索を依頼

any や可変Mapを使用しない。

Titleの処理担当

層	担当
Validator	前後空白の除去、空文字判定、1~100文字の検証
Usecase	正規化済みTitleとCategoryからFilterHashを生成する
Repository	%、_、\ をSQL LIKE用にEscapeし、GORMのプレースホルダーへ渡す
Controller	Queryを受け取り、ValidatorとUsecaseへ渡すだけ

Repositoryへ渡す Title は、SQL用Escape前の値とする。

ListPublic

次を一括取得する。

- Video ID
- Title
- Description
- Category
- CreatedAt
- Author ID
- Author Name
- Output Video Object Key
- Thumbnail Object Key
- LikeCount
- IsLiked
- IsSaved

必須Query条件

```
videos.processing_status = ready
videos.publish_status = published
videos.deleted_at IS NULL
```

OutputMetaが存在しない公開VideoはDB不整合である。

一般Responseへ不完全なVideoを返さず、内部エラーとして記録する。

N+1禁止

次をVideo件数分個別Queryしない。

- Author取得
- Like件数COUNT
- IsLiked存在確認
- IsSaved存在確認
- OutputMeta取得

DB上のRead Modelは1回の主Queryまたは固定回数のBatch Queryで構成する。

VideoLikeRepository Interface

メソッド	責務
Like	UserとVideoを検証・共有ロックし、Like行を冪等作成して、更新後のLike状態を返す
Unlike	公開Videoを確認し、本人のLike行を冪等削除して、更新後のLike状態を返す

Like

- User active確認とVideo公開確認を同一Transactionで行う。
- VideoLike Entityを保存する。

- `uq_video_likes_user_video` 競合を成功扱いへ変換する。
- SQLSTATE、ConstraintName、SQL文をUsecaseへ返さない。
- Like作成または重複確認後、同じTransaction内で対象VideoのLike件数を取得する
- video_id、like_count、is_liked = trueを返す。

Unlike

- 公開条件を満たすVideoだけを対象にする。
- UserIDとVideoIDの両方をWHERE条件にしてLike行を削除する。
- 削除件数0でも成功扱いにする。
- 削除後のLike件数を取得する。
- video_id、like_count、is_liked = falseを返す。

類似検索条件

Repositoryは通常検索結果が0件であることを自ら推測しない。

Usecaseから `SearchMode = similar` を指定された場合だけ、次の条件を追加する。

```
lower(title_query) <% lower(videos.title)
```

類似度は、

```
word_similarity(lower(title_query), lower(videos.title))
```

並び順は、

```
word_similarity(lower(title_query), lower(videos.title)) DESC,
videos.created_at DESC,
videos.id DESC
```

categoryが指定されている場合は、Category完全異t内条件を維持。

Usecase仕様

VideoUsecase

既存公開一覧処理へ検索条件を追加する。

責務

1. Validator済み入力を受け取る。
2. 検索条件をRepository入力へ変換する。
3. Cursorの検索条件識別値を確認する。
4. Repositoryへ公開検索を依頼する。
5. limit + 1件からHasMoreを決定する。
6. 次Cursorを生成する。

- 公開対象だけへ期限付きURLを発行する。
- API Response用Read Modelへ変換する。

行わないこと

- GORM Query作成
- SQL Escape処理
- DB Transaction
- Object Keyの外部返却
- Like行作成
- HTTP Status決定

Cursor

CursorはBackendが発行する不透明文字列とする。

検索Cursorの固定Payloadには次を含める。

項目	内容
ResultType	all、matched、similar
Similarity	similar時の最後の類似度
CreatedAt	最後のVideoの投稿日時
ID	最後のVideo ID
FilterHash	titleとcategoryを含む検索条件識別値

FilterHashは検索語を復元する目的ではなく、Cursorが同じ検索条件で発行されたことを確認するために使用する。

次の場合は400とする。

- 不正Base64
- 不正JSON
- 未定義フィールド
- 不正日時
- IDが0または `math.MaxInt64` 超過
- 現在のTitle・CategoryとFilterHashが一致しない

Frontendは検索条件を変更した場合にCursorを破棄する。

Backendも不一致Cursorを拒否し、別条件のページを混在させない。

検索条件の混在防止には `FilterHash` だけを使用。生成方法は曖昧にせず、

1. trim済みTitleとCategoryを固定構造体へ格納する
2. `encoding/json` でJSON化する
3. JSONのbyte列をSHA-256でHash化する
4. 16進小文字64文字をCursorへ保存する

VideoLikeUsecase

メソッド	責務
Like	Likeを冪等作成し、更新後の <code>video_id</code> 、 <code>like_count</code> 、 <code>is_liked</code> を返す
Unlike	Likeを冪等削除し、更新後の <code>video_id</code> 、 <code>like_count</code> 、 <code>is_liked</code> を返す

Like

1. 認証User IDとVideo IDを受け取る。
2. IDが0でないことを再確認する。
3. Repositoryの用途別Transactionを呼ぶ。
4. 重複Likeは成功として扱う。
5. Repositoryから更新後の`video_id`、`like_count`、`is_liked`を受け取る。
6. Controllerへ返す。

Unlike

1. 認証User IDとVideo IDを受け取る。
2. Repositoryへ本人関係の削除を依頼する。
3. Repositoryから更新後の`video_id`、`like_count`、`is_liked`を受け取る。
4. Controllerへ返す。

Controller仕様

VideoController

既存 `ListReels` を使用し、Query指定時に検索として動作させる。

別のSearchControllerは作らない。

メソッド	内容
ListReels	<code>title</code> 、 <code>category</code> 、 <code>limit</code> 、 <code>cursor</code> を受け取り、公開一覧または検索結果を返す
Detail	既存詳細へ <code>like_count</code> と <code>is_liked</code> を追加する

ListReelsが行うこと

1. Queryを固定Request構造体へBindする。
2. Optional AuthからViewer User IDを取得する。
3. Validatorへ入力を渡す。
4. Usecaseを呼ぶ。
5. 200 Responseを返す。

ListReelsが行わないこと

- SQL生成
- `%` や `_` のEscape
- CategoryのDB検索

- Like COUNT
- Cursor用WHERE生成
- 期限付きURLの保存

VideoLikeController

メソッド	HTTP	内容
Like	PUT	いいねを冪等作成し、更新後のLike状態を 200 OK で返す
Unlike	DELETE	いいねを冪等解除し、更新後のLike状態を 200 OK で返す

Controllerは認証UserをRequest Bodyから受け取らない。

Auth Middlewareが設定した認証Contextだけを使用する。

Middleware仕様

Optional Auth Middleware

検索を含む **GET /videos** と公開詳細へ適用する。

Authorization Header	処理
なし	ゲストとして継続する
正常なToken	User IDをContextへ設定する
不正・期限切れToken	401。ゲスト扱いにしない
TokenVersion不一致	401
suspended User	401

Auth Middleware

次へ適用する。

- **PUT /videos/:video_id/like**
- **DELETE /videos/:video_id/like**

activeなUserだけをControllerへ通す。

AdminはログインUser機能を利用できるため、activeなadminもLike可能とする。

CSRF

いいねAPIはAuthorization HeaderのAccess Tokenで認証する。

Refresh Token Cookieだけで実行しないため、Refresh・Logout用CSRF Middlewareは適用しない。

Rate Limit

要件定義でRedis Token Bucketの必須対象として指定されているのは、会員登録、ログイン、Token再発行、動画投稿関連APIである。

本編では検索・いいね専用の新しいRate Limit値を推測して追加しない。

代わりに次で負荷を制限する。

- limit最大100
- Cursorページング
- 部分一致用GIN索引

- Frontendは明示的な検索Submitで呼び出す
- 入力変更中の不要RequestをAbortする
- LikeはDB UNIQUEで重複を抑止する

将来Search・Like用Rate Limitを追加する場合は、識別単位、容量、補充速度を別途要件化してから既存Redis Token Bucketへ追加する。

Body Limit

本編APIは次のとおりRequest Bodyを使用しない。

- GET検索
- PUT Like
- DELETE Unlike

JSON Bodyを受け取るRouteへ変更しない。

共通Body Limit Middlewareが全Routeへ適用されている場合は、その設定をそのまま使用する。

Validator仕様

検索Query

項目	必須	条件
title	任意	指定された場合はtrim後1~100文字
category	任意	brewing、roasting、latte_art、beans、equipmentのいずれか
limit	任意	未指定20、1~100
cursor	任意	Base64 Raw URL形式、JSON構造、CreatedAt、ID、FilterHashの形式を検証する。最初のページでは指定しない

title

- Query自体が未指定なら検索条件なしとする。
- title= または空白だけが明示指定された場合は400とする。
- 前後空白を除去する。
- HTML Sanitizerで文字を削除しない。
- XSSは表示時の文字列描画で防ぐ。
- SQL EscapeはRepositoryで行う。
- 100文字を超える場合は400とする。

category

- 表示名ではなくCategoryCodeを受け取る。
- 大文字表記や未定義文字列を自動変換しない。
- 未定義値は400とする。

limit

値	処理
未指定	20
1〜100	許可
0、負数、101以上	400
数字以外	400

Cursor

CursorはBackendが発行する不透明文字列とする。

検索Cursorの固定Payloadには次を含める。

項目	型	内容
ResultType	固定文字列型	<code>all</code> 、 <code>matched</code> 、 <code>similar</code> のいずれか
Similarity	<code>float32</code>	<code>similar</code> の場合だけ使用する最後の <code>word_similarity</code> 値。0.6以上1.0以下
CreatedAt	<code>time.Time</code>	最後のVideoの投稿日時
ID	<code>uint64</code>	最後のVideo ID
FilterHash	<code>string</code>	titleとcategoryを含む検索条件識別値

`all` と `matched` ではSimilarityを0とする。

`similar` ではSimilarity、CreatedAt、IDの3項目を使用してKeyset Paginationを行う。

類似検索の次ページ条件は概念上、次の順序とする。

1. 前ページ最後のSimilarityより小さい
2. Similarityが同じ場合はCreatedAtが古い
3. SimilarityとCreatedAtが同じ場合はIDが小さい

Validatorの処理

1. Cursorが未指定の場合は最初のページとして扱う。
2. Base64 Raw URL EncodingとしてDecodeする。
3. 専用Cursor構造体へJSON Decodeする。
4. JSONの不明フィールドを拒否する。
5. `created_at` が有効な日時であることを確認する。
6. `created_at` をUTCとして扱う。
7. `id` が1以上か確認する。
8. `id` が `math.MaxInt64` 以下か確認する。
9. `result_type` が `all`、`matched`、`similar` のいずれかであることを確認する。
10. `similar` の場合は `similarity` が0.6以上1.0以下であることを確認する。
11. `all` または `matched` の場合は `similarity` が0であることを確認する。
12. `filter_hash` が16進小文字64文字か確認する。
13. 不正なCursorは `400 Bad Request` とする。

Validatorは、現在の検索条件とFilterHashが一致するかまでは判定しない。

現在のTitle・CategoryからFilterHashを生成し、Cursor内のFilterHashと比較する処理はUsecaseが担当する。

Cursorの内部項目

項目	型	条件
<code>result_type</code>	<code>string</code>	<code>all</code> 、 <code>matched</code> 、 <code>similar</code> のいずれか
<code>similarity</code>	<code>float32</code>	<code>similar</code> では0.6以上1.0以下。 <code>all</code> と <code>matched</code> では0
<code>created_at</code>	<code>time.Time</code>	有効なUTC日時
<code>id</code>	<code>uint64</code>	1以上、 <code>math.MaxInt64</code> 以下
<code>filter_hash</code>	<code>string</code>	SHA-256を16進小文字で表した64文字

Video ID

既存のVideo ID検証を再利用する。

- 数字である
- 1以上
- `math.MaxInt64` 以下
- 負数、0、上限超過を拒否する

Validatorが行わないこと

- DB検索
- Video公開確認
- User active確認
- Like存在確認
- Like件数取得
- Transaction
- HTTP Response作成

Router・API仕様

API一覧

HTTP	URL	内容	認証	Rate Limit	CSRF
GET	<code>/videos</code>	公開一覧・公開検索	Optional Auth	追加なし	不要
GET	<code>/videos/:video_id</code>	公開動画詳細	Optional Auth	追加なし	不要
PUT	<code>/videos/:video_id/like</code>	いいね	Access Token	追加なし	不要
DELETE	<code>/videos/:video_id/like</code>	いいね解除	Access Token	追加なし	不要

Router接続

URL	Middleware	Controller
<code>GET /videos</code>	Optional Auth	<code>VideoController.ListReels</code>
<code>GET /videos/:video_id</code>	Optional Auth	<code>VideoController.Detail</code>

URL	Middleware	Controller
PUT /videos/:video_id/like	Auth	VideoLikeController.Like
DELETE /videos/:video_id/like	Auth	VideoLikeController.Unlike

Like Routeは `:video_id/like` として明示的に登録する。

API Request ・ Response

公開一覧 ・ 検索

Request

```
GET /videos?title=ハンドドリップ&category=brewing&limit=20&cursor=...
```

Request Queryは変更不要。

```
GET /videos?title=ハンドドリップ&category=brewing&limit=20&cursor=...
```

Responseへ `result_type` を追加する。

```
{
  "items": [],
  "result_type": "similar",
  "next_cursor": null,
  "has_more": false
}
```

Cursorには検索条件とresult_typeが含まれる。
 内容を変更せず、同じtitle・category条件で使用する。
 similar結果のCursorをmatchedまたはallの取得へ使用しない。

Query

name	required	type	description
title	false	string	動画タイトルの部分一致検索語。前後空白を除去し、大文字・小文字を区別しない。指定時は1~100文字
category	false	string	定義済みCategoryCodeとの完全一致
limit	false	integer	1回の取得件数。省略時20、最大100
cursor	false	string	前回Responseの <code>next_cursor</code> 。内容を変更せず同じ検索条件で指定する

Response

```
{
  "items": [
    {
      "id": 125,
      "title": "ハンドドリップの蒸らし方",
      "description": "蒸らし時間の違いを紹介します",
      "category": "brewing",
      "author": {
```

```

      "id":10,
      "name":"山田 太郎"
    },
    "playback_url":"期限付き再生URL",
    "thumbnail_url":"期限付きサムネイルURL",
    "like_count":12,
    "is_liked":true,
    "is_saved":false,
    "created_at":"2026-07-16T11:50:00Z"
  }
],
"next_cursor":null,
"has_more":false
}

```

Responseルール

- `items` は必須。0件は空配列。
- `next_cursor` は次ページがなければnull。
- `has_more` は次ページがある場合だけtrue。
- `like_count` は0以上。
- ゲストの `is_liked` と `is_saved` はfalse。
- Object Keyは返さない。
- 署名付きURLは成功Responseにだけ返し、エラーResponseへ含めない。

いいね

Request

```

PUT /videos/125/like
Authorization: Bearer <access_token>

```

Request Bodyは使用しない。

Response

```

{
  "video_id":125,
  "like_count":12,
  "is_liked":true
}

```

状況	Status
新規Like作成	200 OK
既にLike済み	200 OK

いいね解除

Request

```
DELETE /videos/125/like
Authorization: Bearer <access_token>
```

Response

```
{
  "video_id":125,
  "like_count":11,
  "is_liked":false
}
```

状況	Status
Like関係を削除	200 OK
Like関係が存在しない	200 OK
Videoが一般公開対象外	404 Not Found

HTTP Status • Domain Error

HTTP	条件
200 OK	公開一覧・検索、Like、Unlike成功、幂等再送
400 Bad Request	title、category、limit、cursor、Video ID不正
401 Unauthorized	未認証、不正Token、期限切れ、TokenVersion不一致、suspended User
404 Not Found	Like・Unlike対象が存在しない、または公開対象外
500 Internal Server Error	DB、Cursor、Storage URL発行等の内部失敗

重複Likeは409へしない。

一般公開対象外のVideoについて、private、hidden、削除済み、存在しないを区別するメッセージを返さない。

セキュリティ仕様

認証・認可

- 検索はゲスト利用可能。
- LikeとUnlikeはAccess Token必須。
- User IDをPath、Query、Bodyから受け取らない。
- Auth MiddlewareのUser IDだけを使用する。
- suspended UserをControllerへ通さない。
- AdminはログインUser機能としてLikeを利用できる。

SQL Injection

- title、category、cursor値をSQLへ直接連結しない。
- GORMのプレースホルダーを使用する。
- ORDER BYは固定する。
- 検索対象列をRequestから受け取らない。

- `%`、`_`、`\`をRepositoryでEscapeする。
- `pg_trgm`用SQL断片は固定値だけを使用する。

XSS

- title、description、author.nameをReactの通常テキストとして表示する。
- `dangerouslySetInnerHTML` を使用しない。
- 検索語を検索結果へ強調表示する場合も、HTML文字列を組み立てない。
- URL QueryをDOMへHTMLとして挿入しない。

CSRF

- Like APIはAuthorization HeaderのAccess Tokenで認証する。
- Refresh Token CookieをLike認証へ使用しない。
- Refresh・Logout用CSRF Middlewareは適用しない。

Command Injection

本編はFFmpeg等の外部コマンドを実行しない。

検索語やVideo IDを外部コマンドへ渡さない。

秘密情報の非公開

次をResponseとFrontendログへ含めない。

- SQL
- DB制約名
- Stack Trace
- Object Key
- Storage Credential
- 署名用秘密鍵
- Access Token
- Refresh Token
- Cookie
- Password
- PasswordHash

監査ログ

検索・Likeは管理者による状態変更ではないため、`admin_audit_logs`へ保存しない。

HTTP Request ID、User ID、Route、処理結果は既存の構造化ログへ記録する。

Access Tokenや署名付きURLをログへ記録しない。

Frontend仕様

SearchPage

Route

```
/search
```

検索条件はURL Queryへ保持する。

```
/search?title=ハンドドリップ&category=brewing
```

これにより再読み込み、戻る、共有で同じ条件を復元できる。

表示項目

項目	内容
タイトル入力	任意。最大100文字
カテゴリー選択	全カテゴリーまたは5つのCategory
検索ボタン	明示Submitで検索する
検索結果	VideoCard一覧
追加読込	has_moreがtrueの場合だけ表示
0件表示	「該当する動画はありません」
エラー表示	共通ApiErrorの安全なメッセージ

検索操作

1. 初期表示時にURL Queryを読む。
2. Queryをフォーム初期値へ反映する。
3. Submit時にtitleをtrimする。
4. 空titleはQueryから除去する。ただし空白だけを送信しない。
5. 条件変更時はcursorを破棄する。
6. URL Queryを更新する。
7. 先頭ページを取得する。
8. 次ページ取得時は同じtitle、categoryとnext_cursorを送る。
9. 新しい検索開始時に古いRequestをAbortする。
10. 最新Request以外のResponseを画面へ反映しない。

入力ごとの自動検索は行わない。

不要なAPI連打と古いResponseの上書きを避ける。

LikeButton

表示状態

状態	表示
ゲスト	未選択アイコン、Like件数
ログイン・未Like	未選択アイコン、Like件数

状態	表示
ログイン・Like済み	選択済みアイコン、Like件数
API処理中	ボタン連打を一時無効化
API失敗	元の状態へ戻し、安全なエラーを表示

ゲスト操作

ゲストがLikeボタンを押した場合、Like APIを呼ばない。

現在のページを戻り先としてログイン画面へ遷移する。

```
/login?redirect=<現在のPathとQuery>
```

Open Redirectを防ぐため、redirectは同一アプリ内の相対Pathだけを許可する。

外部URLは受け付けない。

ログインUser操作

1. 現在状態を保持する。
2. ボタンを一時無効化する。
3. 未LikeならPUT、Like済みならDELETEを呼ぶ。
4. PUTまたはDELETEのResponseからvideo_id、like_count、is_likedを受け取る。
5. Responseの値を対象Videoの画面状態へ反映する。
6. 追加の状態取得APIは呼ばない。
7. 失敗時は操作前表示へ戻す。
8. 401時は既存認証処理へ委譲する。
9. 404時は対象Videoを公開画面から外すか再取得する。

GET /videos と GET /videos/:id に含まれる like_count と is_liked を使用する。

既存画面への接続

画面	変更
ReelPage	LikeButtonとLike件数を表示する
VideoDetailPage	LikeButtonとLike件数を表示する
SearchPage	検索結果各VideoへLikeButtonを表示する
ナビゲーション	/search への導線を追加する

レスポンス・アクセシビリティ

- LikeButtonへ button 要素を使用する。
- aria-pressed へis_likedを反映する。
- アイコンだけで意味を伝えず、アクセシブルなラベルを設定する。
- 処理中は disabled を設定する。
- キーボードEnter・Spaceで操作できる。

- 件数が狭い画面で重ならないようにする。
- 検索フォームで不要な横スクロールを発生させない。

OpenAPI仕様

GET /videos

summary

公開動画一覧または公開動画検索

description

titleとcategoryが未指定の場合は、公開条件を満たす動画を投稿日時の新しい順で取得する。

いずれかが指定された場合は通常検索を行う。

titleは大文字・小文字を区別しない部分一致、categoryは定義済みコードとの完全一致である。
両方指定時はAND条件で絞り込む。

titleを含む通常検索が0件の場合は、同じ公開条件を維持したままtitleの類似検索へ切り替える。
categoryも指定されている場合は、そのcategory内だけを類似検索する。

通常検索に一致する動画がある場合は類似動画を混在させない。

類似候補も存在しない場合は空配列を返し、無条件の公開動画一覧へ差し替えない。

Responseの結果typeは、条件未指定時はall、通常一致時はmatched、類似検索時はsimilarとなる。

title description

動画タイトルへ適用する部分一致検索語。

前後空白を除去した1~100文字を受け付ける。

とはワイルドカードではなく通常文字として扱う。

category description

定義済みCategoryCodeとの完全一致条件。

省略時はカテゴリーで絞り込まない。

cursor description

前回Responseで返された次ページ取得用Cursor。

内容を変更せず、同じtitle・category条件で指定する。

検索条件が変わった場合は使用しない。

PUT /videos/{video_id}/like

summary

公開動画へいいねする

description

activeな認証Userと公開VideoのLike関係を作成する。

既にLike済みの場合も新しい行を作らず204を返す。

private、hidden、処理未完了、削除済みVideoは一般公開対象外として404を返す。

DELETE /videos/{video_id}/like

summary

自分のいいねを解除する

description

認証User本人と指定VideoのLike関係を物理削除する。

関係が存在しない場合も204を返す。

他UserのLike関係は削除しない。

Backendテスト

Entity Test

対象	確認内容
VideoLike生成	UserID、VideoID、CreatedAtが正しい
UserID 0	拒否する
VideoID 0	拒否する
更新	更新操作を持たない

Migration Test

対象	確認内容
video_likes作成	カラム、FK、UNIQUE、INDEXが存在する
複合UNIQUE	同じUser・Videoを2件作れない
別User	同じVideoへ作成できる
別Video	同じUserが作成できる
pg_trgm	Extensionが利用可能になる
title索引	公開条件付きGIN索引が存在する
Down	本編索引・テーブルを逆順で削除できる
類似検索の索引	類似検索のQueryで使用する索引が存在する
Word Similarity基準	pg_trgm.word_similarity_threshold`が0.6である
類似検索Index	<code>idx_videos_public_title_trgm</code> を <code><%</code> による類似検索で使用できる

Repository Test ・ 検索

ケース	期待結果
条件なし	公開Videoだけを新しい順で全て取得
通常一致あり	一致したVideoだけを取得し、類似Videoを混在させない
通常一致が0件	類似検索を実行

ケース	期待結果
類似度順	類似度の高いVideoから取得する
同一の類似度	CreatedAt DESC、ID DESCになる
類似候補なし	空の配列になる
private類似候補	取得しない
hidden類似候補	取得しない
deleted類似候補	取得しない
similar Cursor	重複・欠落なく次のページを取得する
別result_type Cursor	400相当へ変換できる
Title完全一致	取得できる
Title途中一致	取得できる
大文字・小文字差	区別せず取得できる
Category一致	指定Categoryだけ取得
Title+Category	両条件一致だけ取得
Title一致・Category不一致	取得しない
private	取得しない
hidden	取得しない
deleted	取得しない
processing	取得しない
failed	取得しない
% 入力	文字%を含むTitleだけ対象になる
_ 入力	文字_を含むTitleだけ対象になる
SQL Injection文字列	SQLとして実行されない
limit + 1	HasMore判定用件数を取得する
Cursor	重複・欠落なく次ページを取得する
別条件Cursor	400相当へ変換できる
Like 0件	like_count 0
Like複数	正しい件数
Guest	is_liked false、is_saved false
Login User	本人is_liked、is_savedが正しい
Query回数	Video件数に比例して増加しない
類似度0.6以上	類似候補として取得する
類似度0.6未満	類似候補として取得しない
類似度順	word_similarity が高いVideoから取得する
同一類似度	CreatedAt DESC、ID DESCになる
長いTitleの類似度	検索語に近い連続部分があれば取得できる
Categoryつきの類似検索	指定Category内だけを取得する

Repository Test ・ Like

ケース	期待結果
active User＋公開Video	Like作成成功

ケース	期待結果
同じLike再送	行数1件、成功
同時Like	行数1件、両Requestを成功扱い
suspended User	作成拒否
private Video	作成拒否
hidden Video	作成拒否
deleted Video	作成拒否
processing Video	作成拒否
管理者非公開と同時に	hidden確定後に新規行を作成しない
User停止と同時に	suspended確定後に新規行を作成しない
Unlike	本人行を物理削除
Unlike再送	成功
他Userの行	削除しない
hidden後Unlike	公開対象外として拒否し、既存Like行を削除しない
DBエラー	SQL・制約名をDomain外へ返さない

Usecase Test

対象	確認内容
検索条件	正規化済み条件をRepositoryへ渡す
Cursor	同じFilterHashだけ許可する
HasMore	limit + 1から正しく決定する
URL発行	公開対象だけへ発行する
Like	更新後のlike_countとis_liked = trueを返す
重複Like	現在のlike_countとis_liked = trueを返す
Unlike	更新後のlike_countとis_liked = falseを返す
Unlike再送	現在のlike_countとis_liked = falseを返す
非公開	Not Foundへ統一する
条件なし	通常の公開一覧だけを取得する
通常一致あり	類似検索Repositoryを呼ばない
通常一致0件+Category0件	類似検索Repositoryを呼ぶ
titleなし+Category0件	類似検索を呼ばず空の配列を返す
mached	result_type = matchedを返す
similar	result_type = similarを返す
all	result_type = allを返す
titleあり+通常一致0件	<code>SearchMode = similar</code> で類似検索Repositoryを呼ぶ
類似検索結果あり	<code>result_type = similar</code> を返す
類似検索結果なし	<code>result_type = similar</code> と空配列を返す

Validator Test

ケース	期待結果
title未指定	許可

ケース	期待結果
title 1文字	許可
title 100文字	許可
title 101文字	400相当
title空文字明示	400相当
title空白だけ	400相当
Category 5値	許可
Category未定義	400相当
limit未指定	20
limit 1・100	許可
limit 0・101・非数	400相当
Cursor不正	400相当
Video ID不正	400相当

Controller Test

ケース	期待Status
Like成功	200、更新後状態を返す
Like再送	200、現在状態を返す
Unlike成功	200、更新後状態を返す
Unlike再送	200、現在状態を返す
未認証Like	401
suspended Like	401
非公開Video Like	404
非公開Video Unlike	404
内部DB失敗	500、内部情報なし

Frontendテスト

SearchPage

ケース	確認内容
初期表示	公開動画一覧を表示する
通常一致	一致結果を表示する
類似Fallback	近い動画と案内文を表示する
類似候補なし	該当する動画はありませんと表示する
Cursor	同じ検索条件とresult_typeで次ページを取得する
Title検索	APIへtitleを送る
Category検索	APIへcategoryを送る
複合検索	両方を送る
条件変更	cursorを破棄する
次ページ	同じ条件とnext_cursorを送る
0件	空状態を表示する

ケース	確認内容
古いResponse	最新検索結果を上書きしない
Enter	Submitできる
101文字	Client側で拒否するがBackend検証にも依存する
XSS文字列	HTMLとして実行しない

最終動作

入力	通常結果	動作
条件なし	—	全公開動画
titleあり	1件以上	一致動画だけ
titleあり	0件	title類似検索
title+category	0件	同じcategory内でtitle類似検索
categoryのみ	0件	空配列
類似候補	0件	空配列

LikeButton

ケース	確認内容
Guest	件数表示、クリックでLoginへ遷移、APIを呼ばない
未Like User	PUTを呼ぶ
Like済みUser	DELETEを呼ぶ
処理中	二重クリックを防ぐ
成功	PUTまたはDELETEのResponseに含まれるlike_countとis_likedを表示へ反映し、追加APIを呼ばない
失敗	元の状態へ戻す
401	既存認証処理へ委譲する
404	非公開化されたVideoを再取得・除外する
aria-pressed	状態と一致する

結合・E2Eテスト

検索フロー

1. brewing、roasting等の公開Videoを複数作成する。
2. private、hidden、deleted、processing Videoも作成する。
3. Guestで条件なし一覧を取得する。
4. Title途中一致で検索する。
5. Categoryで検索する。
6. TitleとCategoryで複合検索する。
7. 公開Videoだけが返ることを確認する。
8. Cursorで次ページを取得する。
9. 条件を変更して古いCursorを指定し、400になることを確認する。
10. LikeCountとGuest状態を確認する。

11. Login Userで同じ検索を行い、本人状態を確認する。

Likeフロー

1. active Userでログインする。
2. 公開VideoへLikeする。
3. 同じPUTを再送する。
4. DB関係行が1件だけであることを確認する。
5. 一覧、検索、詳細でLikeCountとIsLikedを確認する。
6. Unlikeする。
7. 同じDELETEを再送する。
8. DB関係行が0件であることを確認する。
9. 別UserがLikeし、件数がUserごとの関係数になることを確認する。
10. 管理者がVideoをhiddenへ変更する。
11. 検索結果から消えることを確認する。
12. hidden中の新規Likeが404になることを確認する。
13. 既存Like行が保持されていることを確認する。
14. 管理者がpublishedへ戻す。
15. 検索結果へ戻り、既存LikeCountが復元されることを確認する。
16. Userをsuspendedへ変更する。
17. 既存Access TokenでもLike APIが401になることを確認する。

競合E2E

- 同一Userから同一Videoへ同時PUTし、行が1件だけになる。
- 複数Userから同一Videoへ同時PUTし、User数分だけ作成される。
- Like作成と管理者hideを同時実行し、不正な公開判定にならない。
- Like作成とUser停止を同時実行し、停止確定後のLikeを作成しない。
- Unlikeを複数回同時実行しても500にならない。

管理者が動画を `hidden` へ変更した場合、検索結果と保存一覧から除外するが、既存Like行は削除しない。公開再開後は既存Likeを再利用する。

ブラウザ確認

起動条件

- PostgreSQL起動済み
- Migration実行済み
- `pg_trgm` 有効化済み
- Object Storage起動済み
- Backend起動済み
- Frontend起動済み

- 公開Video、private Video、hidden Videoの確認用データあり
- active User、suspended User、adminの確認用アカウントあり

ゲスト検索

1. Logout状態で /search を開く。
2. Titleを入力して検索する。
3. Title途中の語でも一致することを確認する。
4. Categoryを指定する。
5. 複合条件で絞り込めることを確認する。
6. hidden、private、deleted Videoが表示されないことを確認する。
7. LikeCountが表示されることを確認する。
8. Likeボタンを押す。
9. Like APIが呼ばれずLogin画面へ遷移することを確認する。
10. Login後に元の検索URLへ戻れることを確認する。

ログインUser

1. active UserでLoginする。
2. 検索結果へ本人のLike状態が表示されることを確認する。
3. Likeする。
4. 件数と状態が更新されることを確認する。
5. 連打してもDBへ重複作成されないことを確認する。
6. Unlikeする。
7. 再送してもエラー表示にならないことを確認する。
8. リール、詳細、検索結果で状態が整合することを確認する。

管理者非公開との結合

1. Userが公開VideoへLikeする。
2. Adminが対象Videoをhiddenへ変更する。
3. 検索結果から消えることを確認する。
4. Like行がDBに残ることを確認する。
5. Adminがpublishedへ戻す。
6. 検索結果へ戻り、以前の件数が表示されることを確認する。

suspended User

1. active中にLoginする。
2. AdminがUserをsuspendedへ変更する。
3. 既存画面からLikeを試す。
4. 401となり、新しいLike行が作成されないことを確認する。
5. Guestとして自動継続しないことを確認する。

対象ブラウザ

- Chrome
- Safari

PC幅とスマートフォン幅の両方で、検索フォーム、結果一覧、Likeボタンが操作可能であることを確認する。